STOP COLLECTING EVERYTHING

# Vollständige Installationsanleitung

## OpenTelemetry Astronomy Shop auf macOS mit Podman

Prometheus • Jaeger • Grafana • Load Generator • Feature Flags • Sampling-Vorbereitung

**Referenzsystem: Diese Anleitung bildet die auf Marcus' Apple-M4-Mac getestete Umgebung ab: Podman 6.1.1, Docker Compose 5.5.1 und OpenTelemetry Demo 3.1.0-48-gfdbd62ca.**

*Stand: 13. Oktober 2026*

# 1. Ziel und Aufbau

Die Anleitung führt von einem vorbereiteten Mac bis zu einem vollständig laufenden OpenTelemetry Astronomy Shop mit Observability-Backends. Danach ist die Umgebung für Baseline-Messungen, Probabilistic Sampling, Tail Sampling und den späteren MCP-/KI-Advisor vorbereitet.

**Wichtig: Prometheus, Jaeger, Grafana, OpenTelemetry Collector, Load Generator und die Shop-Services werden nicht separat auf macOS installiert. Sie laufen als Container in der Podman Machine.**

## Datenfluss:

```
macOS
 -> Podman + Podman Machine
 -> Docker Compose Provider
 -> OpenTelemetry Demo
    -> Astronomy Shop / Load Generator / flagd
    -> OpenTelemetry Collector
    -> Jaeger / Prometheus / Grafana
    -> später: MCP Sampling Advisor und KI
```

# 2. Getestetes Referenzsystem

| Komponente | Getesteter Stand |
|---|---|
| macOS | 26.6.2, Build 25G83 |
| Architektur | Apple M4 / arm64 |
| Arbeitsspeicher | 16 GB |
| Podman Engine | 6.1.1 |
| Podman Machine | applehv, rootless, user-mode networking |
| Podman Machine Ressourcen | 5 CPUs, 5722 MiB RAM, 100 GiB Disk |
| Compose Provider | Docker Compose 5.5.1 |
| Git | 2.39.5 (Apple Git-154) |
| Python / pip | 3.13.16 / 26.2.1 |
| Node.js / npm / npx | 26.5.0 / 11.17.0 / 11.17.0 |
| Demo Git-Stand | 3.1.0-48-gfdbd62ca |
| Demo Commit | fdbd62ca1ccd98d1752724315994bd2bf67a68db |
| IMAGE_VERSION | 3.1.0 |
| OTel Java Agent | 2.31.1 |

# 3. Was auf dem Mac installiert wird

- Podman Desktop einschließlich Podman CLI
- Docker Compose als externer Compose-Provider
- Git
- Python 3
- Node.js einschließlich npm und npx

## Nicht separat installieren

- Prometheus - wird als Container gestartet
- Jaeger - wird als Container gestartet
- Grafana - wird als Container gestartet
- OpenTelemetry Collector - wird als Container gestartet

- Load Generator - wird als Container gestartet
- PostgreSQL - wird als Container gestartet
- Valkey - wird als Container gestartet
- OpenSearch - wird als Container gestartet
- flagd und die Astronomy-Shop-Microservices - wird als Container gestartet

# 4. Podman Desktop und Podman 6.1.1 installieren

Für die getestete Umgebung wird Podman Desktop über den macOS-DMG-Installer installiert. Im Onboarding werden Podman Engine, Podman CLI, Podman Machine und die Compose CLI eingerichtet.

## Nach der Installation prüfen:

```
podman --version
```

Erwarteter Referenzwert:

```
podman version 6.1.1
```

**Versionshinweis: Podman 6.1.1 ist der auf deinem Mac getestete Stand. Eine neuere Version kann funktionieren, ist mit dieser Anleitung aber nicht 1:1 validiert.**

# 5. Podman Machine konfigurieren

Auf macOS laufen Linux-Container in einer virtuellen Maschine. Prüfe zuerst, ob bereits eine Machine existiert:

```
podman machine list
```

## Nur wenn noch keine Machine existiert:

```
podman machine init \
 --cpus 5 \
 --memory 5722 \
 --disk-size 100

podman machine start
```

## Wenn podman-machine-default bereits existiert:

```
podman machine list
podman machine start podman-machine-default
```

**Nicht erneut initialisieren: Bei der Meldung podman-machine-default already exists nicht nochmals podman machine init ausführen. Die bestehende Machine starten.**

## Prüfen:

```
podman machine inspect podman-machine-default
podman info
podman run --rm quay.io/podman/hello
```

# 6. Docker Compose Provider einrichten

Deine Umgebung verwendet nicht podman-compose, sondern Docker Compose als externen Provider unter /usr/local/bin/docker-compose. Podman ruft diesen Provider über podman compose auf.

## In Podman Desktop:

- Settings öffnen
- Resources auswählen
- Compose öffnen
- Setup ausführen

## Danach prüfen:

```
which docker-compose
docker-compose version
podman compose version
```

Erwartete Referenzausgabe:

```
/usr/local/bin/docker-compose
Docker Compose version v5.5.1
```

**Hinweis: Die Meldung „Executing external compose provider /usr/local/bin/docker-compose“ ist in diesem Setup korrekt. podman-compose muss nicht zusätzlich installiert werden.**

# 7. Git, Python und Node.js prüfen

## Git:

```
git --version
which git
```

Referenz: git 2.39.5 unter /usr/bin/git.

## Python:

```
python3 --version
which python3
python3 -m pip --version
```

Referenz: Python 3.13.16 und pip 26.2.1.

## Node.js:

```
node --version
npm --version
npx --version
which node
```

Referenz: Node.js 26.5.0, npm/npx 11.17.0, installiert unter /opt/homebrew/bin.

**Einordnung: Python und Node.js sind für den Shop-Containerstart nicht erforderlich. Sie werden später für Hilfsbefehle, den MCP Sampling Advisor und den MCP Inspector benötigt.**

# 8. OpenTelemetry Demo herunterladen

```
mkdir -p ~/Documents/otel-cost-workshop
cd ~/Documents/otel-cost-workshop
git clone https://github.com/open-telemetry/opentelemetry-demo.git
cd opentelemetry-demo
```

## Den getesteten Git-Stand auschecken:

```
git checkout fdbd62ca1ccd98d1752724315994bd2bf67a68db
git switch -c podman-ai-sampling-lab
```

Prüfen:

```
git remote -v
git branch --show-current
git rev-parse HEAD
git describe --tags --always
```

Erwartet: Branch podman-ai-sampling-lab, Commit fdbd62ca… und Beschreibung 3.1.0-48-gfdbd62ca.

# 9. Umgebungs- und Compose-Dateien prüfen

```
ls -la \
 compose.yaml \
 compose.observability.yaml \
 compose.extras.yaml \
 .env
```

Versionsvariablen prüfen:

```
grep -E "IMAGE_VERSION|DEMO_VERSION|OTEL.*VERSION" .env
```

Getestete Werte:

```
IMAGE_VERSION=3.1.0
DEMO_VERSION=latest
OTEL_JAVA_AGENT_VERSION=2.31.1
```

Falls .env.override noch nicht existiert, anlegen:

```
touch .env.override
```

# 10. Podman-spezifische Grafana-Anpassung

In der getesteten Umgebung benötigte Grafana ein höheres Memory-Limit. Erstelle im Repository-Hauptverzeichnis compose.podman.yaml:

```
services:
 grafana:
  deploy:
   resources:
    limits:
     memory: 512M
```

**Konsequenz: compose.podman.yaml muss in jedem Start-, Status-, Log- und Stop-Befehl berücksichtigt werden.**

# 11. Ports und Speicher kontrollieren

```
df -h ~
lsof -i :8080
lsof -i :9090
lsof -i :8000
```

Für den Stack werden vor allem Port 8080 für den Frontend Proxy und Port 9090 für Prometheus verwendet. Port 8000 wird später für den MCP Advisor benötigt.

# 12. OpenTelemetry Demo starten

```
cd ~/Documents/otel-cost-workshop/opentelemetry-demo

podman compose \
 --project-name otel-demo \
 --env-file .env \
 --env-file .env.override \
 -f compose.yaml \
 -f compose.observability.yaml \
 -f compose.extras.yaml \
 -f compose.podman.yaml \
 up \
 --force-recreate \
 --remove-orphans \
 --detach
```

**Wichtig: Nicht nur podman compose up mit compose.yaml ausführen. Die Observability- und Extras-Dateien starten Jaeger, Prometheus, Grafana und weitere benötigte Komponenten.**

# 13. Containerstatus prüfen

```
podman ps --format \
 "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Der Start kann abgeschlossen gelten, wenn insbesondere diese Container laufen:

- frontend-proxy
- frontend
- load-generator
- otel-collector
- jaeger
- prometheus
- grafana
- flagd
- flagd-ui
- astronomy-db
- valkey-cart

Collector-Fehler prüfen:

```
podman logs otel-collector 2>&1 \
 | grep -iE "error|invalid|failed|unknown"
```

Keine Ausgabe bedeutet bei diesem Filter, dass keine passende Fehlermeldung gefunden wurde.

# 14. Getestete Container-Komponenten

| Komponente | Getestetes Container-Image |
|---|---|
| OpenTelemetry Collector | opentelemetry-collector-contrib:0.160.0 |
| Prometheus | quay.io/prometheus/prometheus:v3.13.3 |
| Jaeger | quay.io/jaegertracing/jaeger:2.21.0 |
| Grafana | docker.io/grafana/grafana:13.2.1 |
| PostgreSQL | docker.io/library/postgres:18.6 |
| Valkey | ghcr.io/valkey-io/valkey:9.1.2-alpine3.24 |
| flagd | ghcr.io/open-feature/flagd:v0.16.0 |
| Demo Services | ghcr.io/open-telemetry/demo:latest-* |

**Reproduzierbarkeit: Die Demo-Services verwenden in deinem Lauf latest-* Tags. Der Git-Commit ist festgehalten, die Demo-Container sind ohne zusätzliche Image-Digests jedoch nicht vollständig eingefroren.**

# 15. Benutzeroberflächen öffnen

| Komponente | Adresse |
|---|---|
| Astronomy Shop | http://localhost:8080/ |
| Jaeger | http://localhost:8080/jaeger/ui/ |
| Grafana | http://localhost:8080/grafana/ |
| Prometheus | http://localhost:9090/ |
| Load Generator | http://localhost:8080/loadgen/ |
| Feature Flags | http://localhost:8080/feature |

**Grafana: Grafana über den Frontend Proxy unter /grafana/ öffnen. Nicht den dynamischen Host-Port aus podman port grafana als Browser-Adresse verwenden.**

# 16. Prometheus prüfen

```
curl -s http://localhost:9090/-/healthy
```

Danach in der Prometheus-Oberfläche nach Span-Metriken suchen:

```
{__name__=~".*span.*calls.*"}
{__name__=~".*span.*duration.*"}
```

Erst nachdem die realen Metrik- und Labelnamen bestätigt wurden, werden die Baseline-PromQL-Abfragen ausgeführt.

# 17. Jaeger prüfen

Öffne den Shop, führe eine Produkt-, Warenkorb- und Checkout-Journey aus und öffne anschließend Jaeger. Suche nach Services wie frontend, checkout oder product-catalog. Öffne einen Trace und prüfe, ob mehrere Spans und Services sichtbar sind.

```
podman ps | grep jaeger
podman logs jaeger --tail 100
```

# 18. Grafana prüfen

```
podman exec grafana \
  curl -s http://127.0.0.1:3000/api/health
```

Erwarteter Kern der Antwort:

```
{
  "database": "ok",
  "version": "13.2.1"
}
```

Danach Grafana über http://localhost:8080/grafana/ öffnen.

# 19. Load Generator und Feature Flags prüfen

## Load Generator:

```
podman ps | grep load-generator
podman logs load-generator --tail 100
```

Die Demo 3.1.0 verwendet den Locust-basierten Load Generator. Für Sampling-Vergleiche sollte bei jeder Variante dieselbe Last und derselbe Zeitraum verwendet werden.

## Feature Flags:

```
podman ps | grep flagd
```

Über die Feature-Flag-Oberfläche können später kontrollierte Fehler- oder Slow-Szenarien erzeugt werden. Nach dem Test die Flags wieder zurücksetzen.

# 20. Ende-zu-Ende-Test

- Shop unter http://localhost:8080/ öffnen
- Mehrfach Produkt -> Warenkorb -> Checkout ausführen
- Jaeger öffnen und einen Trace finden
- Prometheus öffnen und Span-Metriken finden
- Grafana Health API prüfen
- Load Generator prüfen
- Feature Flags öffnen

**Erfolgskriterium: Die Umgebung ist bereit, wenn Shop-Traffic einen Trace im Collector erzeugt, Jaeger diesen Trace anzeigt und Prometheus die dazugehörigen Span Metrics enthält.**

# 21. Baseline-Messung vorbereiten

Wenn traces_span_metrics_calls_total vorhanden ist, können folgende Abfragen als Startpunkt dienen:

## Gesamtrate:

```
sum(rate(traces_span_metrics_calls_total[5m]))
```

### Top 10 Operationen:

```
topk(
 10,
 sum by (service_name, span_name) (
  rate(traces_span_metrics_calls_total[5m])
 )
)
```

### Fehlerquote:

```
100 *
sum by (service_name, span_name) (
 rate(traces_span_metrics_calls_total{
  status_code="STATUS_CODE_ERROR"
 }[5m])
)
/
sum by (service_name, span_name) (
 rate(traces_span_metrics_calls_total[5m])
)
```

### p95-Latenz:

```
histogram_quantile(
 0.95,
 sum by (le, service_name, span_name) (
  rate(
   traces_span_metrics_duration_milliseconds_bucket[5m]
  )
 )
)
```

**Testregel: Zwischen Baseline, 50 %, 10 %, Tail Sampling und KI-Policy nur das Sampling ändern. Last, Dauer, Journey und Prometheus-Zeitraum müssen gleich bleiben.**

## 22. Stack stoppen und erneut starten

### Stoppen, ohne Volumes zu löschen:

```
podman compose \

--project-name otel-demo \

--env-file .env \

--env-file .env.override \

-f compose.yaml \

-f compose.observability.yaml \

-f compose.extras.yaml \
```

```
-f compose.podman.yaml \

up --force-recreate --remove-orphans --detach
```

### Erneut starten:

```
podman machine start
cd ~/Documents/otel-cost-workshop/opentelemetry-demo

podman compose \
 --project-name otel-demo \
 --env-file .env \
 --env-file .env.override \
 -f compose.yaml \
 -f compose.observability.yaml \
 -f compose.extras.yaml \
 -f compose.podman.yaml \
 up --force-recreate --remove-orphans --detach
```

## 23. Finaler Go/No-Go-Test

```
echo "=== HOST ==="
podman --version
podman compose version
git --version
python3 --version
node --version

echo "=== MACHINE ==="
podman machine list

echo "=== CONTAINERS ==="
podman ps --format \
 "table {{.Names}}\t{{.Status}}\t{{.Image}}"

echo "=== PROMETHEUS ==="
curl -s http://localhost:9090/-/healthy

echo "=== GRAFANA ==="
podman exec grafana \
 curl -s http://127.0.0.1:3000/api/health

echo "=== COLLECTOR ERRORS ==="
podman logs otel-collector 2>&1 \
 | grep -iE "invalid|failed|unknown" | tail -20
```

### Go/No-Go-Checkliste:

- ☐ Podman 6.1.1 verfügbar
- ☐ Podman Machine läuft
- ☐ Docker Compose 5.5.1 wird als Provider erkannt
- ☐ Alle zentralen Container laufen
- ☐ Shop erreichbar
- ☐ Jaeger erreichbar und Traces sichtbar
- ☐ Prometheus erreichbar und Span Metrics vorhanden
- ☐ Grafana Health = database ok
- ☐ Load Generator erreichbar

- ☐ Feature Flags erreichbar
- ☐ Collector zeigt keine Konfigurationsfehler

# 24. Nächster Abschnitt des Labs

Nach erfolgreicher Installation folgt der eigentliche Optimierungsablauf:

```
Baseline messen
 -> 50 % Probabilistic Sampling
 -> 10 % Probabilistic Sampling
 -> Tail Sampling
 -> Prometheus-Daten über MCP bereitstellen
 -> KI erzeugt Sampling-Empfehlung
 -> Human Review
 -> Before/After messen
```

**Ziel: Nicht nur weniger Telemetrie sammeln, sondern nachweisen, dass Fehler, langsame Traces und geschäftskritische Abläufe weiterhin sichtbar bleiben.**